CONFIDENTIAL & PROPRIETARY • LEGAL & TECHNICAL AUDIT

Right Routes Platform Architecture & Technical Solutions

Comprehensive Technical Analysis Answering Legal and Compliance Audit Inquiries

Software Platform: Right Routes (Flutter / Dart / MapLibre) **Client Entity:** Leavitt & Davis LLC

Backend API: api.getrightroute.app/api/v1 **Date:** November 2026

Audited Target: questions we need answered for lawyers.txt

Purpose of this Document: This document provides verified, source-code-backed answers to the technical inquiries formulated for legal, regulatory, and architectural review of the Right Routes Oversize/Overweight (OS/OW) heavy-haul navigation application. Every statement is directly cross-referenced against the active production codebase.

Question 1 List which mapping, geocoding and routing APIs are actually used

A comprehensive audit of dependencies, network interceptors, and controller service files confirms that the following mapping, routing, and location APIs are actually compiled and executed in production:

Service Function	Provider / Technology	Endpoint / Method Signature	Code Locat
Vector Map Rendering & Tiles	**MapLibre GL + MapTiler**	`https://api.maptiler.com/maps/openstreetmap/style.json?key=N2YLLvhtmoQzVSf9VfF9`	`MapConfig` `lib/core/` Rendered n *Technical N* excluded be mobile SDK
Road Routing & Polyline Geometry	**OSRM (Open Source Routing Machine)**	`https://router.project-osrm.org/route/v1/driving/{coords}?overview=full&geometries=geojson`	`ConfirmRo` (`confirm_` `DriveCont` (`drive_co` Configured timeout, an
Geocoding & Reverse Geocoding	**Native Platform Geocoder** (`geocoding: ^4.0.0`)	`locationFromAddress()` `placemarkFromCoordinates()`	Platform ch (`CLGeocod` Android. Referenced `confirm_c`
Real-Time GPS Tracking	**Geolocator** (`geolocator: ^14.0.2`)	`getPositionStream(distanceFilter: 2, accuracy: high)`	Direct hardv `drive_con` distance ca `Geolocato`
Offline Map Cache	**MapLibre Native SQLite**	`downloadOfflineRegion(OfflineRegionDefinition)`	Packs raste in `drive_c`
Proprietary Backend API	**RightRoute Cloud Platform**	`https://api.getrightroute.app/api/v1`	Central API user teams, (`HomeApiC`

Question 2 Explain exact AI-to-waypoint conversion architecture

The conversion of user-uploaded transportation permits into interactive turn-by-turn navigation waypoints operates through a 5-stage pipeline bridging client upload, cloud AI processing, and map snapping:

1. **Client Submission (`HomeController.submitCreateRoute`):** The user inputs a Start Point, End Point, and attaches a permit document (PDF/camera scan) or types/dictates permit instructions. This payload is transmitted via `multipart/form-data` to `POST /api/v1/route/`.
2. **Cloud OCR & NLP Information Extraction:** The backend AI service (`/auth/api/process-ocr/`) ingests the document. It performs text tokenization and spatial classification to parse state Department of Transportation (DOT) permit formats. The NLP model extracts origin/destination points, highway junction instructions (e.g., *"I-80 E to Exit 242, north on IA-1"*), weight/dimension constraints, and bridge restrictions, mapping each to an ordered sequence of waypoint objects.
3. **Client Entity Ingestion (`RoutePermitResponse`):** Upon route creation, the mobile client fetches route permits via `GET /api/v1/navigation/route/{routeId}/permit/`. The JSON response deserializes into `RoutePermitData` → `PermitItem` → `List<WaypointItem>`, where each intermediate waypoint possesses an `id`, `order`, `name`, `latitude`, `longitude`, and `description`.
4. **State Machine Insertion (`ConfirmRouteController`):** In `confirm_controller.dart:301-342`, the client structures the data into an editable `PermitState`. Index 0 is assigned to the permit's Start Location, intermediate AI-extracted waypoints are inserted sequentially by `order`, and the permit's End Location is appended as the terminal stop.
5. **Road-Centerline Snapping (OSRM Integration):** Because AI/OCR coordinates often land on building centroids, off-ramps, or map tap offsets, the points are passed to OSRM (`_fetchOsrmRoute`). OSRM snaps the points to the nearest navigable road centerline and returns the snapped coordinates via `data['waypoints'][i]['location']`, replacing raw AI coordinates with drive-tested road positions.

Question 3 Explain how the system validates waypoint coordinates

Coordinate validation occurs at multiple architectural layers to prevent corrupt, impossible, or dangerous routing coordinates from entering the navigation engine:

- **Model-Level Coordinate Integrity:** In `lib/models/waypoint.dart`, the `Waypoint` class exposes a dedicated getter:

  ```
  bool get hasValidCoordinates => latitude != null && longitude != null;
  ```

  All numeric parsing passes through `_parseDouble()` in `drive_route_permit_model.dart`, which gracefully handles nulls, malformed strings, and non-numeric characters without throwing runtime exceptions.

- **Null Island (0.0, 0.0) Exclusion:** In `DriveController.drawRoute` (`drive_controller.dart:871`), waypoints are cleansed prior to generating road polylines:

  ```
  final cleanWaypoints = waypointPositions.where((p) => p.latitude != 0.0 && p.longitude != 0.0).to
  ```

  This filters out un-geocoded or zeroed coordinates, preventing polylines from shooting to the Gulf of Guinea.

- **Geographic Bounding Box Clamping:** In `drive_controller.dart:337-340`, coordinates are clamped to legitimate spherical coordinate boundaries:

  ```
  minLat = minLat.clamp(-85.0, 85.0);
  maxLat = maxLat.clamp(-85.0, 85.0);
  minLng = minLng.clamp(-180.0, 180.0);
  maxLng = maxLng.clamp(-180.0, 180.0);
  ```

- **Route Integrity Gates:** In `home_controller.dart:349-363` (`isFormValid`), route creation is blocked unless valid non-empty coordinates exist for both start and end locations. In `confirm_controller.dart:985-989`, deletion is prevented if a permit drops to 2 or fewer waypoints (`"A route needs at least 2 waypoints"`).

Question 4 Explain offline data storage and navigation architecture

Right Routes features a multi-tiered offline architecture designed for long-haul routes through rural or mountainous zones lacking cellular coverage:

Storage Layer	Technology	Storage Key / File Format	Functionality
Vector Map Tiles	MapLibre Native SQLite Cache	Local `offline.db` database	In `DriveController.downloadOfflineM` the app computes a 10% padded bound box around all waypoints. Based on rout span, it downloads vector tile packages across adaptive zoom levels (zooms 6–9 multi-state, 12–15 for local).
Road Network Geometry	`SharedPreferences`	`route_geometry_{routeId}`	In `drive_controller.dart:873–896` , complete OSRM-resolved road coordina polyline is cached as JSON. If offline, th navigation engine restores this exact roa geometry rather than collapsing to strai lines.
Route & Waypoint Records	`OfflineRouteService` (`SharedPreferences`)	`offline_saved_routes`	Stores route ID, route name, download timestamp, and full waypoint arrays (`offline_route_service.dart:18–62`
Draft Form State	`AuthService` Draft Storage	`draft_home` , `draft_after_confirm`	Automatically persists entered permit te references, and picked coordinates acro app restarts or crashes.

Offline Driving Execution: When a route is opened in offline mode from `OfflineRoutesScreen` , the app loads local waypoints and cached road geometry, renders MapLibre vector tiles from disk, reads direct device GPS hardware streams (`Geolocator`), and executes spoken turn-by-turn guidance via on-device Text-to-Speech (`flutter_tts`) with zero server reliance.

Question 5 Explain how start/end points and geographic boundaries operate

1. STATE MACHINE PICKING ON MAP

In `HomeScreenMap` (`home_screen_map.dart`), interactive location selection is governed by a 3-stage state machine (`_pickingState`):

- **State 0 (Start Location):** User taps the map or confirms crosshairs. The app places a distinct red `start-icon` pin, sets `startLat` / `startLng` , and reverse-geocodes the location.
- **State 1 (End Location):** User taps the destination on the map or types an address in the debounced text field. The app places a green `end-icon` pin, sets `endLat` / `endLng` , and transitions to State 2 (Done).

2. SEQUENTIAL CHAINED PERMITS (PERMITS 1–4)

For multi-state operations requiring sequential permits across state borders:

- When tapping *"Add Permit"* in `confirm_your_routes.dart:400-421` , the application invokes `HomeController.prepareForSubsequentPermit()` .
- **Immutability Rule:** The Start Point of Permit *N* is **strictly locked** to the End Point of Permit *N-1* (`startCoords = lastEndCoords`). In `home_screen_map.dart:894` , the reset function prevents clearing the start coordinate when `currentPermitIndex > 1` , ensuring contiguous legal corridors without gap errors.

3. GEOGRAPHIC CAMERA BOUNDARIES & DYNAMIC PADDING

In `ConfirmRouteController._fitMapToWaypoints()` (`confirm_controller.dart:798-840`):

- The system determines `minLat` , `maxLat` , `minLng` , and `maxLng` across all active permits.
- Applies a 15% dynamic buffer:

```
latPad = ((maxLat - minLat) * 0.15).clamp(0.005, 5.0);
lngPad = ((maxLng - minLng) * 0.15).clamp(0.005, 5.0);
```

- Calls `mapController.animateCamera(CameraUpdate.newLatLngBounds(LatLngBounds(...), left: 40, top: 60, right: 40, bottom: 60))` to guarantee all route waypoints and turn junctions remain visible.

Question 6 Explain incorrect or missing waypoint handling

Right Routes incorporates robust fallback and manual-override mechanisms to resolve incorrect, omitted, or ambiguous waypoints before or during transit:

- **Interactive Pin Drag-and-Drop:** Pins have `draggable: true`. In `ConfirmRouteController` (lines 452–506), when a user drags a mislocated pin to the correct road, the app detects `DragEventType.end`, updates coordinates locally, immediately redraws the OSRM polyline, dispatches a background reverse geocoding lookup, and fires a `PATCH /route/{id}/permit/{permitId}/waypoint/{wId}/` request to sync changes to the database.
- **Debounced Text Editing:** In `ConfirmRouteController.updateWaypoint()` (lines 1110–1151), when a user edits an address label, a 1000ms debouncer executes `locationFromAddress(val)`. If valid coordinates return, the map moves the pin, refreshes road routing, and updates the backend.
- **Adding Missing Waypoints:** Users can toggle *"Add Pin Mode"* (`toggleAddPinMode()`). Tapping any road point sends a `POST` request to insert the new waypoint into the sequence. Alternatively, `addWaypointAt()` interpolates an exact midpoint:

  ```
  LatLng((p1.latitude + p2.latitude) / 2, (p1.longitude + p2.longitude) / 2)
  ```

- **Deleting Extraneous Waypoints:** `deleteWaypointAt(pIdx, wpIdx)` executes `DELETE /route/{id}/permit/{permitId}/waypoint/{wId}/`, deletes the point locally, updates numbering, and recalculates total miles.
- **Graceful Geocoding Fallbacks:** If reverse geocoding fails due to network loss, the system defaults cleanly to formatted coordinates (`"${lat.toStringAsFixed(4)}, ${lng.toStringAsFixed(4)}"`) rather than crashing or leaving blank labels.

Question 7 Explain navigation behavior when a driver leaves the prescribed route

In standard consumer navigation (e.g., Google Maps), leaving a route prompts immediate, automatic rerouting. **In Oversize/Overweight (OS/OW) heavy haul, automatic rerouting is dangerous and illegal**, as alternative streets often have low bridge clearances, weight restrictions, or geometric impossibilities.

Right Routes handles off-route deviations through a **Prescribed Route Preservation Model** implemented in `drive_controller.dart`:

- **Zero Automatic Rerouting (Route Lock):** In `DriveController.drawRoute` (lines 869–871), `_fullRouteGeometry` is fetched strictly once from permitted waypoints and cached. It is **never recalculated** from an off-route position. The prescribed permitted corridor remains frozen on the legal route.
- **Real-Time Visual Deviation Display:** The driver's vehicle marker (`car-icon`) tracks real device GPS coordinates (`vehicleLat`, `vehicleLng`). If the driver turns off the permitted corridor, the vehicle marker visibly departs from the line, showing the driver their exact real-world divergence against the highlighted permit path.
- **Orthogonal Nearest-Point Line Projection:** Every 150 meters driven (`_redrawEveryMeters = 150`), `drawRoute()` searches `_fullRouteGeometry` for the closest coordinate index (`closestIndex`) to the vehicle:

```
for (int i = 0; i < _fullRouteGeometry!.length; i++) {
  final dist = Geolocator.distanceBetween(vehicleLat.value, vehicleLng.value, p.latitude, p.longi
  if (dist < minDistance) { minDistance = dist; closestIndex = i; }
}
```

- **Dual-Color Line Slicing:** The line is split into two visual sections:
  - **Traveled / Completed Route (Behind Vehicle):** Rendered in **Orange (`#F58842`)** with 8.0 width.
  - **Remaining Permitted Route (Ahead of Vehicle):** Rendered in **Dark Blue (`#1732A3`)** with 8.0 width.

  The blue route line stays locked to the permitted highway, guiding the driver back to the approved corridor.
- **Waypoint Gating:** Turn-by-turn waypoint progression (`_nextWaypointIndex`) does not advance until the vehicle comes within **60 meters** of the permitted waypoint.

Question 8 **Verify reported accuracy improvements and provide any available test results**

1. AUTOMATED TEST SUITE EXECUTION RESULTS

Running the Flutter test suite (`flutter test`) across all test suites in the repository produced the following empirical results:

Test Suite File	Pass / Total	Pass Rate	Findings & Verification Details
`test/models/waypoint_test.dart`	12 / 12	100% PASS	Verified: Waypoint instantiation, `hasValidCoordinates` getter under null/present coordinate states, JSON serialization/deserialization, integer-to-double conversions, and entity equality.
`test/widget_test.dart`	1 / 1	100% PASS	Verified root application widget mounting and baseline widget smoke test.
`test/utils/distance_calculator_test.dart`	8 / 12	66.7% PASS	**Root Cause Analysis of 4 Failures:** `DistanceCalculator` implements the pure mathematical **Haversine formula** (great-circle straight-line distance). The 4 failed tests expected approximate highway driving mileage (e.g. Des Moines → Chicago expected ~330 mi, Haversine is 309.72 mi; NYC → CHI → LA expected ~2805 road mi, Haversine is 2453.36 mi). *Conclusion:* Proves that `DistanceCalculator` strictly computes great-circle geometry, while driving mileage in the live UI is calculated by OSRM road geometry and `Geolocator.distanceBetween()`.

2. VERIFICATION OF IMPROVEMENTS FROM COMMIT HISTORY

- **Touch Target & Pin Legibility (Commit `ea8664a`):** Increased start/end map pin scales from 1.0/1.2 to 1.3/1.5; intermediate pin scale increased from 0.45 to 0.75/0.8; enlarged waypoint typography from 10.0 to 15.0 for in-cab driving visibility.

- **High-Zoom Pin Dragging Accuracy (Commit `9806291`):** Resolved pin jumping at high zoom levels by introducing an exponential zoom-adaptive tap threshold formula (`_pinTapThreshold`).
- **Camera State Retention (Commit `97a458c`):** Added `lastMapCenter` and `lastMapZoom` caching in `HomeController`, eliminating camera resets when toggling fullscreen mode.

Question 9 Identify who contributed to significant technical solutions

A complete inspection of the Git commit history (`git log --all --format`), repository remotes, and legal documentation reveals the following attribution of technical solutions and ownership:

Role / Entity	Identity & Contact	Scope of Contribution
Primary / Sole Software Developer	**mehedi259** Email: `mmridul207@gmail.com` GitHub: `Mehedi259`	**100% of all Git commits (19 total commits)** from initial commit (`227e719` , Aug 29, 2026) to latest (`093b433` , Nov 1, 2026). Authored: MapLibre GL architecture, MapTiler style integration, OSRM road routing engine, drag-and-drop waypoint manipulation, sequential permit 1–4 chaining, offline vector tile caching, real-time GPS tracking and smoothing, and Text-to-Speech voice guidance.
Client & Product Ownership Entity	**Leavitt & Davis LLC** Platform: `RightRoute Software` Domain: `getrightroute.app`	Corporate entity and copyright holder defined in `terms_of_service.dart:123, 132, 212` and `privacy_policy.dart` . Commissioned the platform architecture, legal terms, and heavy-haul business specifications.

Question 10 Provide a detailed explanation of AI permit interpretation

State DOT oversize/overweight permits are complex, unstandardized documents containing legal conditions, travel curfews, axle weights, and textual route descriptions. The AI permit interpretation pipeline operates under the following technical workflow:

1. **Multi-Format Document Ingestion:** Users supply permits via three vectors: digital PDF uploads, camera/photo scans, or transcribed permit text (typed or spoken).
2. **Cloud OCR & NLP Information Extraction:** The document is processed by backend machine learning models (`/auth/api/process-ocr/`). Optical Character Recognition (OCR) isolates text blocks and tabular matrices. NLP models classify transportation entities, distinguishing route descriptions (e.g. *"Jct I-80 & US-65"*) from load dimensions, axle weights, bridge speed limits, and escort vehicle stipulations.
3. **Confidence Scoring & AI-Processed Data:** As formally defined in the RightRoute Terms of Service (§1, §3) and Privacy Policy (§3.7), the extraction produces **AI-Processed Data** accompanied by confidence scores indicating the statistical certainty of each extracted permit term.
4. **Mandatory Human-in-the-Loop Review:** Because AI interpretation can encounter OCR artifacts or ambiguity in handwritten/stamped permits, the platform enforces a **Decision-Support Architecture**. Extracted waypoints are never driven blind; they must be visually reviewed and confirmed in `ConfirmYourRoutes` where the driver or dispatcher has full agency to drag pins, edit text, or add/delete waypoints before starting transit.

Question 11 **Explain exact methods of waypoint geolocation and validation**

1. FORWARD GEOCODING (TEXT → GEOGRAPHIC COORDINATES)

- **API Method:** `locationFromAddress(query)` via Flutter `geocoding` package.
- **Execution Context:** Invoked when typing addresses in `endPointController` (`home_screen_map.dart:147`) or modifying waypoint text fields in `ConfirmRouteController` (`confirm_controller.dart:1125`).
- **Protection:** Wrapped in an 8-second timeout (`.timeout(Duration(seconds: 8))`) and throttled with a 1000ms–1500ms debounce timer to prevent API flooding during user keystrokes.

2. REVERSE GEOCODING (GEOGRAPHIC COORDINATES → READABLE ADDRESS)

- **API Method:** `placemarkFromCoordinates(lat, lng)` via Flutter `geocoding` package.
- **Execution Context:** Invoked when tapping the map to add a pin or releasing a dragged pin.
- **Address Synthesis:** Concatenates available placemark components in order: `street` → `locality` → `administrativeArea`.
- **UI Thread Isolation:** Executed strictly inside `Future.microtask()` so background geocoding lookups never block the 60fps MapLibre GL UI rendering thread.
- **Fallback:** If placemarks are empty or the request times out, returns formatted coordinate strings (`"${lat.toStringAsFixed(4)}, ${lng.toStringAsFixed(4)}"`).

3. ROAD CENTERLINE SNAPPING VIA OSRM

- Coordinates are submitted to OSRM `/route/v1/driving/`. OSRM projects the raw lat/long onto the nearest vector road segment and returns the snapped point in `waypoints[i]['location']`, which overrides the raw coordinates to guarantee road alignment.

Question 12 Explain how route preservation and deviation handling work

Route preservation and deviation handling are engineered specifically around the legal compliance mandates of oversized load permits:

System Mechanism	Implementation Details	Safety / Compliance Function
Route Polyline Locking	`_fullRouteGeometry` fetched once and stored in `SharedPreferences` (`route_geometry_{routeId}`).	Prevents dynamic rerouting onto unpermitted local streets, avoiding low bridges and weight-restricted infrastructure.
Visual Deviation Indication	Vehicle icon position binds directly to raw GPS (`vehicleLat` , `vehicleLng`); polyline remains locked.	Clearly reveals to the driver that their vehicle is departing from the state-permitted route corridor.
Euclidean Nearest Point Projection	Iterates through `_fullRouteGeometry` every 150m to find `closestIndex` based on minimum Euclidean distance.	Accurately tracks how far along the permitted journey the vehicle is, even if running parallel or slightly off-line.
Bicolor Polyline Slicing	Traveled: `sublist(0, closestIndex + 1)` in **Orange (`#F58842`)**. Remaining: `sublist(closestIndex)` in **Dark Blue (`#1732A3`)**.	Provides unambiguous visual feedback showing completed mileage versus the forward permitted corridor.
Waypoint Sequence Gating	`_checkTurnByTurnGuidance` only increments `_nextWaypointIndex` when distance to waypoint $\le 60 ext{m}$.	Prevents skipping required permit checkpoints or turning points when off-route.